Heurísticas y búsqueda local: demostraciones y ejemplos para el pizarrón

Departamento de Computación, FCEyN, UBA — 2do cuatrimestre de 2026

1. Búsqueda local: terminación y óptimo local

Proposición 1. *Sea $s_0 \in \mathsf{Sols}_{válidas}$ y consideremos el algoritmo que, mientras exista $s' \in \mathcal{N}(s)$ con $c(s') < c(s)$, hace $s \leftarrow s'$. Entonces el algoritmo termina, y la solución que devuelve es un óptimo local para $\mathcal{N}$.*

Demostración. Sea $s_0, s_1, s_2, \ldots$ la sucesión de soluciones visitadas. Por construcción, $c(s_0) > c(s_1) > c(s_2) > \cdots$, es decir, la sucesión de costos es *estrictamente decreciente*. Como $\mathsf{Sols}_{\text{válidas}}$ es finito, el conjunto de valores $\{c(s) : s \in \mathsf{Sols}_{\text{válidas}}\}$ es finito, digamos de K elementos. Una sucesión estrictamente decreciente sobre un conjunto de K valores tiene a lo sumo K términos (no puede repetir ninguno), así que el ciclo se ejecuta a lo sumo $K - 1$ veces y el algoritmo termina.

Al terminar, la condición del ciclo es falsa: no existe $s' \in \mathcal{N}(s)$ con $c(s') < c(s)$. Es decir, $c(s) \leq c(s')$ para toda $s' \in \mathcal{N}(s)$: exactamente la definición de óptimo local. □

Nota: La cota K sobre las iteraciones es real pero, en general, inútil: para el TSP con distancias enteras entre 1 y D, $K \leq nD$, polinomial en n y D; pero con distancias reales arbitrarias K puede ser $|\mathsf{Sols}_{\text{válidas}}|$, exponencial. Y de hecho hay instancias en las que 2-opt hace una cantidad exponencial de iteraciones. Lo que la proposición garantiza es *terminación*, no *eficiencia*; y *óptimo local*, no *óptimo*.

1.1. El óptimo local de 4 reinas

Representación: $s = (s_0, s_1, s_2, s_3)$, donde s_j es la fila de la reina de la columna j. Costo: cantidad de pares (i, j), $i < j$, con $s_i = s_j$ o $|s_i - s_j| = j - i$. Vecindad: mover una reina dentro de su columna (12 vecinas).

La configuración $s = (0, 2, 3, 1)$ tiene costo 1: el único par en conflicto es $(1, 2)$, porque $|s_1 - s_2| = |2 - 3| = 1 = 2 - 1$. Sus 12 vecinas:

Columna	Mover a fila	Costo	Columna	Mover a fila	Costo
0	1	4	2	0	2
0	2	2	2	1	2
0	3	3	2	2	3
1	0	**1**	3	0	3
1	1	2	3	2	3
1	3	2	3	3	3

Ninguna vecina tiene costo < 1: s es un óptimo local que no es solución. La vecina $(0, 0, 3, 1)$ (columna 1 a fila 0) tiene también costo 1 (las reinas de las columnas 0 y 1 quedan en la misma fila): una *meseta*. Una búsqueda local que acepte movimientos de igual costo puede salir por ahí; la versión estricta de la proposición, no.

2. Max-Cut: una búsqueda local que es una 2-aproximación

Dado $G = (V, E)$, una partición (A, B) de V define el corte $\delta(A, B) = \{\{u, v\} \in E : u \in A,\ v \in B\}$. Queremos maximizar $|\delta(A, B)|$. Vecindad: cambiar un vértice de lado.

Teorema 2. *Sea (A, B) un óptimo local para la vecindad «cambiar un vértice de lado». Entonces $|\delta(A, B)| \geq \frac{|E|}{2} \geq \frac{\mathrm{OPT}}{2}$. Además, la búsqueda local que parte de cualquier partición y se mueve mientras el corte crezca hace a lo sumo $|E|$ iteraciones.*

Demostración. Para cada vértice v, sean $c(v)$ la cantidad de aristas incidentes a v que cruzan el corte y $i(v)$ la cantidad que no cruzan (tienen los dos extremos del mismo lado que v). Si cambiamos a v de lado, las $c(v)$ aristas dejan de cruzar y las $i(v)$ pasan a cruzar; el resto no cambia. El nuevo corte tiene $|\delta(A, B)| - c(v) + i(v)$ aristas. Como (A, B) es óptimo local, ese valor es $\leq |\delta(A, B)|$, es decir,

$$i(v) \leq c(v) \qquad \text{para todo } v \in V.$$

Sumamos sobre todos los vértices. Cada arista que cruza el corte se cuenta en $c(v)$ para sus dos extremos, y cada arista que no cruza se cuenta en $i(v)$ para sus dos extremos:

$$\sum_v c(v) = 2\,|\delta(A, B)|, \qquad \sum_v i(v) = 2\,\big(|E| - |\delta(A, B)|\big).$$

Sumando la desigualdad $i(v) \leq c(v)$ sobre v: $2(|E| - |\delta(A, B)|) \leq 2|\delta(A, B)|$, o sea $|\delta(A, B)| \geq |E|/2$. Como ningún corte tiene más de $|E|$ aristas, $\mathrm{OPT} \leq |E|$, y por lo tanto $|\delta(A, B)| \geq \mathrm{OPT}/2$.

Para la cantidad de iteraciones: el corte es un entero entre 0 y $|E|$, y cada iteración lo aumenta en al menos 1. Luego hay a lo sumo $|E|$ iteraciones. □

Nota: La demostración usa *únicamente* que estamos en un óptimo local: no importa desde dónde arrancamos ni en qué orden movimos vértices. Eso es lo que convierte una heurística en un algoritmo de aproximación: un teorema sobre el *resultado*, no sobre el *proceso*. Con pesos en las aristas el mismo argumento da corte $\geq w(E)/2$, pero la cantidad de iteraciones ya no está acotada por $|E|$ (Kleinberg–Tardos 12.4 arregla esto exigiendo mejoras de al menos un factor $(1 + 2\varepsilon/n)$).

3. Vertex cover: 2-aproximación por matching maximal

El algoritmo: $C \leftarrow \emptyset$, $E' \leftarrow E$; mientras $E' \neq \emptyset$, elegir $\{u, v\} \in E'$, agregar u y v a C, y borrar de E' toda arista incidente a u o a v. Llamamos M al conjunto de aristas elegidas.

Lema 3. *M es un matching (sus aristas son disjuntas dos a dos) y C es el conjunto de extremos de M; en particular $|C| = 2|M|$.*

Demostración. Al elegir $\{u, v\}$ se borran de E' todas las aristas incidentes a u o a v; las aristas que se eligen después están en E', así que no tocan ni a u ni a v. Luego dos aristas elegidas no comparten extremos. Como C recibe exactamente los dos extremos de cada arista elegida y estos son todos distintos, $|C| = 2|M|$. □

Lema 4. *C es un cubrimiento por vértices.*

Demostración. Sea $e \in E$. Al principio $e \in E'$ y al final $E' = \emptyset$, así que en algún momento e fue borrada. Una arista se borra sólo cuando es incidente al u o al v recién agregados a C. Luego e tiene un extremo en C. □

Lema 5. *Todo cubrimiento C^* cumple $|C^*| \geq |M|$. En particular* $\mathrm{OPT} \geq |M|$.

Demostración. Para cada arista $e \in M$, C^* contiene al menos un extremo de e; elijamos uno y llamémoslo $\varphi(e)$. Como las aristas de M son disjuntas dos a dos (Lema 3), si $e \neq e'$ entonces $\varphi(e) \neq \varphi(e')$: la función $\varphi : M \to C^*$ es inyectiva, y $|C^*| \geq |M|$. □

Teorema 6. *El algoritmo es una 2-aproximación para vertex cover.*

Demostración. Es polinomial: cada iteración borra al menos una arista, así que hay a lo sumo $|E|$ iteraciones, cada una de costo $O(n)$ con listas de adyacencia (o $O(n+m)$ en total con un poco de cuidado). Devuelve un cubrimiento (Lema 4). Y $|C| = 2|M| \leq 2 \cdot \text{OPT}$ por los Lemas 3 y 5. □

Nota: La cota de OPT es el matching: cualquier cubrimiento necesita un vértice *distinto* por cada arista del matching. Esa es la única información sobre OPT que usamos, y alcanza. Para mostrar que el 2 es ajustado basta un grafo que sea un único matching perfecto (por ejemplo una sola arista): el algoritmo devuelve los $2|M|$ vértices y el óptimo es $|M|$. El ejemplo de las diapositivas (óptimo $\{2, 4, 7\}$, C de tamaño 6) también es ajustado.

4. TSP métrico: 2-aproximación por árbol generador mínimo

Instancia: K_n con $d(u, v) \geq 0$ simétrica y con desigualdad triangular $d(u, w) \leq d(u, v) + d(v, w)$. Algoritmo: calcular un AGM T, recorrerlo en preorden desde un vértice r, devolver el tour H que visita los vértices en ese orden.

Lema 7. $c(T) \leq \text{OPT}$.

Demostración. Sea H^* un tour óptimo. Quitando cualquiera de sus aristas obtenemos un camino que pasa por todos los vértices: un subgrafo conexo, acíclico y generador, es decir un árbol generador T'. Como T es un árbol generador de costo mínimo, $c(T) \leq c(T') = c(H^*) - d(e) \leq c(H^*) = \text{OPT}$, usando $d(e) \geq 0$. □

Lema 8 (recorrido de Euler)**.** *Sea W la secuencia de vértices que visita un DFS de T desde r, anotando un vértice cada vez que se llega a él (al descubrirlo y cada vez que se vuelve a él desde un hijo). Entonces W empieza y termina en r, cada arista de T aparece exactamente dos veces entre vértices consecutivos de W, y por lo tanto $c(W) = 2\,c(T)$.*

Demostración. Por inducción en la estructura del árbol. El DFS desde r baja por la arista (r, h) hacia cada hijo h, recorre recursivamente el subárbol de h (que por hipótesis inductiva usa cada arista de ese subárbol dos veces y vuelve a h) y sube por (h, r). Así cada arista (r, h) se usa exactamente dos veces, una en cada sentido, y las aristas internas de los subárboles, dos veces cada una por hipótesis inductiva. □

Lema 9 (atajos)**.** *Sea $W = w_0, w_1, \ldots, w_k$ un recorrido (cerrado) y sea H la secuencia que resulta de borrar de W toda aparición de un vértice que ya apareció antes (salvo el w_0 final, que cierra el tour). Entonces $c(H) \leq c(W)$.*

Demostración. Basta ver que borrar *una* aparición repetida no aumenta el costo; el resultado sigue aplicándolo una aparición por vez. Si borramos w_i (con $0 < i < k$), el tramo $w_{i-1} \to w_i \to w_{i+1}$, de costo $d(w_{i-1}, w_i) + d(w_i, w_{i+1})$, se reemplaza por la arista $w_{i-1} \to w_{i+1}$, de costo $d(w_{i-1}, w_{i+1})$. Por la desigualdad triangular, $d(w_{i-1}, w_{i+1}) \leq d(w_{i-1}, w_i) + d(w_i, w_{i+1})$. El resto del recorrido no cambia. □

Teorema 10. *En el TSP métrico, $c(H) \leq 2 \cdot \text{OPT}$. El algoritmo es una 2-aproximación.*

Demostración. El preorden del DFS es exactamente lo que queda de W al borrar las apariciones repetidas (un vértice se anota en preorden sólo la primera vez que se lo visita). H pasa por cada vértice exactamente una vez y vuelve a r: es un tour. Encadenando los tres lemas,

$$c(H) \;\le\; c(W) \;=\; 2\,c(T) \;\le\; 2 \cdot \mathrm{OPT}.$$

El AGM se calcula en $O(n^2)$ con Prim sobre K_n; el DFS, en $O(n)$. □

Ejemplo 11. Puntos $0 = (0,0), 1 = (2,3), 2 = (4,0), 3 = (6,3), 4 = (8,0), 5 = (3,-2)$ con distancia euclídea. AGM: aristas $01, 12, 23, 34$ (cada una $\sqrt{13} \approx 3{,}61$) y 25 ($\sqrt{5} \approx 2{,}24$); $c(T) \approx 16{,}66$. DFS desde 0 visitando los hijos en orden creciente: $W = 0\,1\,2\,3\,4\,3\,2\,5\,2\,1\,0$, $c(W) \approx 33{,}32$. Borrando repetidos: $H = 0\,1\,2\,3\,4\,5\,0$, $c(H) \approx 23{,}41$ (los atajos $4 \to 5$ y $5 \to 0$ reemplazan a $4\,3\,2\,5$ y a $5\,2\,1\,0$). El óptimo es $0\,1\,3\,4\,2\,5\,0$, de costo $\approx 21{,}05$. Se verifica $16{,}66 \le 21{,}05 \le 23{,}41 \le 33{,}32$.

Nota: La desigualdad triangular se usa *una sola vez*, en el lema de los atajos. Todo lo demás vale para cualquier función de distancia no negativa. Por eso es la hipótesis que hay que nombrar, y por eso el siguiente teorema no debería sorprender.

5. TSP general: no hay aproximación de factor constante

Teorema 12. *Si $P \neq NP$, para ningún $\rho \ge 1$ constante existe una ρ-aproximación polinomial para el TSP con distancias arbitrarias (no negativas).*

Demostración. Supongamos que existe una ρ-aproximación $\mathcal{A}$ polinomial; vamos a usarla para decidir en tiempo polinomial si un grafo tiene un ciclo hamiltoniano, problema que es NP-completo. Eso implica P = NP.

Dado $G = (V, E)$ con n vértices, construimos una instancia del TSP sobre K_n con

$$d(u,v) = \begin{cases} 1 & \text{si } \{u,v\} \in E, \\ \rho n + 1 & \text{si no.} \end{cases}$$

La construcción es polinomial (hay $\binom{n}{2}$ distancias, y $\rho n + 1$ se escribe con $O(\log n)$ bits).

Si G tiene un ciclo hamiltoniano Z, entonces Z es un tour de la instancia y usa sólo aristas de E, de peso 1: $c(Z) = n$, así que $\mathrm{OPT} \le n$. Como todo tour tiene n aristas de peso ≥ 1, $\mathrm{OPT} = n$, y $\mathcal{A}$ devuelve un tour de costo $\le \rho n$.

Si G no tiene ciclo hamiltoniano, todo tour de la instancia usa al menos una arista que no está en E, de peso $\rho n + 1$, más otras $n - 1$ aristas de peso ≥ 1: $\mathrm{OPT} \ge \rho n + 1 + (n-1) = \rho n + n > \rho n$. Entonces *cualquier* tour, en particular el que devuelve $\mathcal{A}$, tiene costo $> \rho n$.

Luego «correr $\mathcal{A}$ y comparar su costo con ρn» decide ciclo hamiltoniano en tiempo polinomial. □

Nota: Los pesos $\rho n + 1$ violan groseramente la desigualdad triangular (un camino de dos aristas de peso 1 es mucho más barato que la arista directa). El teorema anterior y este son las dos caras de la misma hipótesis. El argumento se llama *gap reduction*: se construye una instancia donde el óptimo es «chico» o «muy grande», sin nada en el medio, y una aproximación distinguiría los dos casos.

6. A*

Grafo dirigido $G = (V, E)$ con pesos $w \ge 0$, origen s, destino t, y $h^*(v) = \mathrm{dist}(v, t)$ (distancia real de v a t; ∞ si no hay camino). Una heurística $h : V \to \mathbb{R}_{\ge 0}$ es *admisible* si $h(v) \le h^*(v)$ para todo v, y *consistente* si $h(t) = 0$ y $h(u) \le w(u,v) + h(v)$ para toda arista (u, v).

Lema 13. *Si h es consistente, entonces es admisible.*

Demostración. Sea v con $h^*(v) < \infty$ (si $h^*(v) = \infty$ no hay nada que probar) y sea $P = v = u_0, u_1, \ldots, u_k = t$ un camino mínimo de v a t, de k aristas. Probamos $h(u_i) \leq \text{dist}(u_i, t)$ por inducción *hacia atrás* en i, de k a 0. Caso base $i = k$: $h(t) = 0 = \text{dist}(t, t)$. Paso: si $h(u_{i+1}) \leq \text{dist}(u_{i+1}, t)$, por consistencia $h(u_i) \leq w(u_i, u_{i+1}) + h(u_{i+1}) \leq w(u_i, u_{i+1}) + \text{dist}(u_{i+1}, t) = \text{dist}(u_i, t)$, donde la última igualdad usa que un subcamino de un camino mínimo es mínimo. Con $i = 0$: $h(v) \leq \text{dist}(v, t) = h^*(v)$. □

Teorema 14. *Si $w \geq 0$ y h es consistente, A* (la versión de las diapositivas: prioridad $f = g + h$, cada vértice se extrae a lo sumo una vez, se detiene al extraer t) devuelve $g(t) = \text{dist}(s, t)$.*

Demostración. Definimos, para cada arista, $w'(u, v) = w(u, v) - h(u) + h(v)$. Por consistencia, $w'(u, v) \geq 0$.

Los caminos mínimos no cambian. Para cualquier camino $P = v_0, v_1, \ldots, v_k$,

$$w'(P) = \sum_{i=0}^{k-1} \big(w(v_i, v_{i+1}) - h(v_i) + h(v_{i+1})\big) = w(P) - h(v_0) + h(v_k),$$

porque la suma telescopa. Para caminos de s a t, $w'(P) = w(P) - h(s) + h(t) = w(P) - h(s)$: todos se desplazan la misma constante $h(s)$, así que P minimiza w' si y sólo si minimiza w, y $\text{dist}'(s, t) = \text{dist}(s, t) - h(s)$. Más en general, para cualquier v, $\text{dist}'(s, v) = \text{dist}(s, v) - h(s) + h(v)$.

A es Dijkstra sobre w'.* Corramos Dijkstra sobre (G, w') desde s, con etiquetas g'. Afirmamos que en todo momento $g'(v) = g(v) - h(s) + h(v)$ para todo v, donde g son las etiquetas de A* sobre (G, w, h), y que ambos algoritmos extraen los vértices en el mismo orden. Inducción en la cantidad de extracciones: al inicio $g'(s) = 0 = g(s) - h(s) + h(s)$ y los demás valen ∞. Si la afirmación vale antes de una extracción, Dijkstra extrae el v de menor $g'(v) = g(v) + h(v) - h(s) = f(v) - h(s)$, que es el de menor $f(v)$: el mismo que A* (con la misma regla de desempate). Al relajar (u, v), Dijkstra compara $g'(u) + w'(u, v) = g(u) - h(s) + h(u) + w(u, v) - h(u) + h(v) = (g(u) + w(u, v)) - h(s) + h(v)$ con $g'(v) = g(v) - h(s) + h(v)$: la comparación es exactamente $g(u) + w(u, v) < g(v)$, la de A*, y la actualización mantiene la relación.

Conclusión. Dijkstra con pesos no negativos es correcto (clase 10): cuando extrae un vértice v, $g'(v) = \text{dist}'(s, v)$, y cada vértice se extrae a lo sumo una vez. Por lo tanto, cuando A* extrae t, $g(t) = g'(t) + h(s) - h(t) = \text{dist}'(s, t) + h(s) = \text{dist}(s, t)$. Además, detenerse al extraer t es correcto porque el valor de un vértice extraído no cambia más. □

Nota: La consistencia es *exactamente* la condición $w' \geq 0$: no es una hipótesis técnica arbitraria, es la hipótesis de Dijkstra en el grafo repesado. Con $h \equiv 0$, $w' = w$ y A* es Dijkstra. La demostración también dice por qué A* puede extraer menos vértices: se detiene al extraer t, y en el grafo repesado los vértices «en la dirección equivocada» tienen dist' grande (porque h es grande) y se extraen después de t.

Con h sólo admisible, y A* pesado (opcional)

Consideramos ahora la versión de A* *con reapertura*: cuando una relajación mejora $g(v)$, v se (re)inserta en la cola aunque ya haya sido extraído. Usamos prioridad $f_\lambda(v) = g(v) + \lambda\, h(v)$ con $\lambda \geq 1$ y h admisible.

Lema 15. *Sea $P = s = v_0, v_1, \ldots, v_k = t$ un camino mínimo de s a t. En todo momento antes de extraer t, existe algún $v_j \in P$ que está en la cola con $g(v_j) = \text{dist}(s, v_j)$.*

Demostración. En todo momento $g(v)$ es el costo de algún camino real de s a v, así que $g(v) \geq \text{dist}(s, v)$ siempre. Sea j el menor índice tal que v_j *nunca* fue extraído con $g(v_j) = \text{dist}(s, v_j)$. Existe,

porque $t = v_k$ todavía no fue extraído. Si $j = 0$: s está en la cola desde el inicio con $g(s) = 0 = \text{dist}(s, s)$, y si ya hubiera sido extraído lo habría sido con ese valor; así que s sigue en la cola. Si $j \geq 1$: v_{j-1} fue extraído con $g(v_{j-1}) = \text{dist}(s, v_{j-1})$; en ese momento se relajó (v_{j-1}, v_j), dejando $g(v_j) \leq \text{dist}(s, v_{j-1}) + w(v_{j-1}, v_j) = \text{dist}(s, v_j)$, y como $g(v_j) \geq \text{dist}(s, v_j)$ siempre, $g(v_j) = \text{dist}(s, v_j)$ desde entonces, y v_j fue insertado en la cola. Si después hubiera sido extraído, lo habría sido con ese valor, contra la elección de j. Luego v_j sigue en la cola con $g(v_j) = \text{dist}(s, v_j)$. □

Teorema 16. *Si $w \geq 0$, h es admisible y $\lambda \geq 1$, la versión con reapertura de A^* con prioridad f_λ devuelve $g(t) \leq \lambda \cdot \text{dist}(s, t)$. Con $\lambda = 1$, devuelve* $\text{dist}(s, t)$.

Demostración. Sea $C^* = \text{dist}(s, t)$ y P un camino mínimo. En el instante en que se extrae t, por el lema hay un $v_j \in P$ en la cola con $g(v_j) = \text{dist}(s, v_j)$, y como t tiene la prioridad mínima,

$$\begin{aligned} g(t) = f_\lambda(t) \leq f_\lambda(v_j) &= \text{dist}(s, v_j) + \lambda\, h(v_j) \leq \text{dist}(s, v_j) + \lambda\, \text{dist}(v_j, t) \\ &\leq \lambda\,(\text{dist}(s, v_j) + \text{dist}(v_j, t)) = \lambda\, C^*, \end{aligned}$$

usando $h(t) = 0$ (admisibilidad en t), $h(v_j) \leq h^*(v_j) = \text{dist}(v_j, t)$ (admisibilidad), $\lambda \geq 1$ y que v_j está sobre un camino mínimo. Con $\lambda = 1$ queda $g(t) \leq C^*$, y como $g(t) \geq C^*$ siempre, $g(t) = C^*$. □

Nota: El teorema no acota cuántas veces se reabre un vértice: con h admisible pero no consistente, puede ser exponencial. La consistencia es la que garantiza «una extracción por vértice». El ejemplo de las diapositivas (s–a–b–t con $h(a) = 4$, $h(b) = 0$) muestra que *sin* reapertura y sin consistencia la respuesta puede ser subóptima: 7 en vez de 5.

7. Respuestas modelo a las preguntas de parcial

1. **Vecindad y óptimo local; terminación.** Una vecindad es una función $\mathcal{N} : \text{Sols}_{\text{válidas}} \to 2^{\text{Sols}_{\text{válidas}}}$ que asigna a cada solución un conjunto de soluciones «cercanas». s es óptimo local si $c(s) \leq c(s')$ para toda $s' \in \mathcal{N}(s)$. La búsqueda local termina porque el costo decrece estrictamente en cada iteración y toma finitos valores (Sección 1). Garantiza devolver un óptimo local para $\mathcal{N}$; no garantiza nada respecto del óptimo global.

2. **Heurística vs. aproximación.** Ambas devuelven una solución factible en tiempo razonable (polinomial, en el caso de la aproximación) sin pretender optimalidad. Una ρ-aproximación viene con una demostración de que $\text{ALG} \leq \rho \cdot \text{OPT}$ en *toda* instancia; una heurística no tiene tal garantía y se evalúa empíricamente. Para el TSP métrico: vecino más cercano es una heurística (puede ser $\Theta(\log n)$ veces el óptimo); el tour del árbol generador mínimo es una 2-aproximación.

3. **Vertex cover.** Algoritmo y demostración en la Sección 3: C = extremos de un matching maximal M; C cubre porque toda arista se borró por tener un extremo en C; $\text{OPT} \geq |M|$ porque todo cubrimiento necesita un vértice distinto por cada arista del matching; $|C| = 2|M| \leq 2\,\text{OPT}$. Es ajustada: en un grafo formado por una sola arista, $|C| = 2$ y $\text{OPT} = 1$.

4. **TSP métrico.** Algoritmo y demostración en la Sección 4: $c(H) \leq c(W) = 2\,c(T) \leq 2\,\text{OPT}$. La desigualdad triangular se usa en el paso $c(H) \leq c(W)$ (los atajos). Sin ella el teorema es falso: por la Sección 5, si P $\neq$ NP no existe ninguna aproximación de factor constante para el TSP general.

5. **Metaheurística y GRASP.** Una metaheurística es una estrategia general, independiente del problema, para guiar heurísticas (construcción, búsqueda local) más allá de un óptimo local. GRASP itera dos fases: (1) construcción greedy randomizada: en cada paso se arma la lista restringida de candidatos $\text{RCL} = \{c : g(c) \leq g_{\text{mín}} + \alpha(g_{\text{máx}} - g_{\text{mín}})\}$ y se elige uno al azar;

(2) búsqueda local desde la solución construida. Se guarda la mejor solución de todas las iteraciones. $\alpha = 0$: greedy puro, determinista, todas las iteraciones dan lo mismo. $\alpha = 1$: construcción aleatoria sin criterio. Valores intermedios balancean calidad y diversidad de las soluciones iniciales; α se calibra experimentalmente.

6. **Admisible, consistente, garantía de A*.** Admisible: $h(v) \leq h^*(v) = \text{dist}(v, t)$ para todo v. Consistente: $h(t) = 0$ y $h(u) \leq w(u, v) + h(v)$ para toda arista. Consistente implica admisible (Lema 13). Con h consistente, A* devuelve la distancia mínima y extrae cada vértice a lo sumo una vez, porque es exactamente Dijkstra sobre los pesos $w'(u, v) = w(u, v) - h(u) + h(v) \geq 0$, que preservan los caminos mínimos $s \to t$ (Teorema 14). Con $h \equiv 0$, A* es Dijkstra.

7. **2-opt y greedy por grado.** 2-opt no es un algoritmo de aproximación para el TSP: es una heurística de búsqueda local; sin desigualdad triangular sus óptimos locales pueden ser arbitrariamente malos, y aun con ella no tiene factor constante. El greedy por grado para vertex cover no es una 2-aproximación: su factor de peor caso es $\Theta(\log n)$; la 2-aproximación es el matching maximal.

8. **Admisible pero no consistente.** Sí, sin reapertura A* puede devolver un camino subóptimo. Ejemplo: vértices s, a, b, t con $w(s, a) = 1$, $w(a, b) = 1$, $w(s, b) = 4$, $w(b, t) = 3$; $h(s) = 5$, $h(a) = 4$, $h(b) = 0$, $h(t) = 0$. h es admisible ($h(a) = h^*(a) = 4$, $h(b) = 0 \leq 3$) pero no consistente ($h(a) = 4 > 1 + h(b)$). A* extrae s; inserta a con $f = 5$ y b con $f = 4$; extrae b con $g(b) = 4$ e inserta t con $f = 7$; extrae a y encuentra $g(b) = 2$, pero b ya está cerrado; extrae t con $g(t) = 7$. El óptimo es $s\, a\, b\, t$, de costo 5.